Assignment 11 — Solutions

RAG & Vector Databases

Part C — Building GenAI Applications · 21 questions with answers

Q1. What is RAG? Explain its architecture (indexing pipeline + retrieval + generation) with a diagram. ★

Answer

RAG (Retrieval-Augmented Generation) gives an LLM access to **external, up-to-date or private knowledge** at query time: relevant passages are retrieved and inserted into the prompt, and the LLM answers using them. It reduces hallucination, enables citations, and avoids retraining when data changes.

Indexing pipeline (offline):

1. **Load** documents (PDFs, web pages, DB records).
2. **Split** into chunks.
3. **Embed** each chunk with an embedding model.
4. **Store** vectors + text + metadata in a **vector database**.

Retrieval + generation (online, per query):

5. Embed the user **query** with the same embedding model.
6. **Retrieve** the top-k most similar chunks (optionally hybrid search + re-ranking).
7. **Augment** the prompt: instructions + retrieved context + question.
8. **Generate** the answer with the LLM (with citations).

Diagram: Docs → Chunker → Embeddings → Vector DB; Query → Embedding → Vector DB search → top-k chunks → Prompt → LLM → Answer.

Q2. RAG vs fine-tuning vs long-context prompting — when would you use each? ★

Answer

- **RAG** – best when the model needs **knowledge**: private/company documents, frequently changing data, need for **citations** and access control. Cheap to update (just re-index). Default choice for Q&A over documents.
- **Fine-tuning** – best for changing **behaviour, style, format or specialized skills** (tone of voice, domain jargon, structured output, classification, a smaller cheaper model for a narrow task). Not good for injecting frequently changing facts; needs training data and retraining.
- **Long-context prompting** – put the whole document(s) directly in the prompt. Good for a **small number of documents** that fit the window (analyzing one contract), and quick prototypes. Expensive and slow at scale, and "lost in the middle" issues.

They can be **combined**: e.g. fine-tune a model for format/domain and use RAG for facts.

Q3. What are embeddings? Name OpenAI embedding models and open-source embedding models. Can you reduce OpenAI embedding dimensions? ★

Answer

Embeddings are dense vectors that represent the meaning of text; similar meanings have nearby vectors. In RAG they are used to find chunks semantically similar to the query.

OpenAI: text-embedding-ada-002 (1536-d, older), **text-embedding-3-small** (1536-d), **text-embedding-3-large** (3072-d).

Open-source: sentence-transformers (all-MiniLM-L6-v2, all-mpnet-base-v2), **BGE** (BAAI bge-small/base/large, bge-m3 multilingual), **E5** (multilingual-e5), GTE, Nomic-embed, Jina embeddings, Instructor, Stella. (Also Cohere embed-v3, Google text-embedding models, Voyage – commercial.)

Reducing OpenAI dimensions: yes – the text-embedding-3 models support a `**dimensions**` parameter (e.g. 3072 → 256 or 1024). They were trained with **Matryoshka representation learning**, so shortened vectors keep most of the quality while saving storage and search cost.

```
client.embeddings.create(model="text-embedding-3-large", input="hello", dimensions=1024)
OpenAIEmbeddings(model="text-embedding-3-large", dimensions=1024)   # LangChain
```

Q4. How do you choose an embedding model (MTEB leaderboard, domain, language, dimensions, cost)?

Answer

- **Benchmark:** check the **MTEB leaderboard** (retrieval score), then **evaluate on your own data** (recall@k on a set of real queries) – this matters most.
- **Domain:** general vs specialized (legal, medical, finance, code); fine-tune an embedding model if needed.
- **Language:** multilingual support (e.g. Hindi/Marathi → bge-m3, multilingual-e5, Cohere multilingual).
- **Dimensions:** more dims = more storage and slower search; 384–1024 is often enough.
- **Max input length:** 512 tokens vs 8k tokens – must match your chunk size.
- **Cost and deployment:** API pricing vs self-hosting; latency; **data privacy** (can data leave your network?).
- Remember: changing the embedding model later means **re-embedding the whole corpus**.

Q5. What are chunking strategies (fixed, recursive, semantic, document-structure based)? How do you choose chunk size? ★

Answer

- **Fixed-size chunking** – split every N characters/tokens with overlap. Simple and fast; can cut sentences.
- **Recursive character chunking** – split on paragraphs → lines → sentences → words to respect natural boundaries (good default).
- **Sentence-based** – group whole sentences up to a size limit.
- **Semantic chunking** – embed sentences and start a new chunk where the meaning changes (embedding similarity drops). Better coherence, more compute.
- **Document-structure based** – split by headings/sections (Markdown, HTML), pages, tables, code functions; keep headings as metadata.
- **Advanced:** parent–child chunks (small for retrieval, large for context), contextual chunk headers, agentic/LLM-based chunking.

Choosing chunk size: depends on the content and questions – typical **256–1,024 tokens with 10–20% overlap**. Small chunks → precise retrieval but missing context; large chunks → more context but noisier embeddings and more tokens. **Experiment** and measure retrieval quality (context recall/precision) on a test set; also respect the embedding model's max input length.

Q6. How are indexes created in vector databases? Explain flat, IVF, PQ and HNSW. ★

Answer

- **Flat (brute-force) index** – compares the query with **every vector**. 100% exact, but O(N) – fine up to ~100k vectors.
- **IVF (Inverted File Index)** – clusters vectors with **k-means** into many lists (centroids). At query time, only the closest `nprobe` clusters are searched → much faster, approximate.
- **PQ (Product Quantization)** – **compresses** vectors: split each vector into sub-vectors and replace each with the ID of its nearest centroid in a small codebook (e.g. 768 floats → 96 bytes). Huge memory savings, approximate distances. Often combined: **IVF-PQ**.
- **HNSW (Hierarchical Navigable Small World)** – a multi-layer **graph** of vectors; search greedily walks the graph. Very fast and high recall; uses more memory. Default in most vector DBs.

These are **Approximate Nearest Neighbour (ANN)** indexes – they trade a little accuracy for large speed gains. (Others: LSH, Annoy trees, ScaNN, DiskANN.)

Q7. How does HNSW (Hierarchical Navigable Small Worlds) work? ★

Answer

HNSW builds a **multi-layer proximity graph**, similar to a skip list:

- **Layer 0** (bottom) contains **all** vectors, each linked to its M nearest neighbours.
- Higher layers contain **progressively fewer** randomly selected nodes with long-range links (like express highways).

Search:

1. Start at an entry point in the **top layer**.
2. **Greedily move** to the neighbour closest to the query until no neighbour is closer.
3. Drop down one layer and repeat from that node.
4. In the bottom layer, do a wider search (beam of size **ef_search**) and return the top-k.

Insertion: a new vector gets a random max layer and is connected to its nearest neighbours in each layer.

Parameters: M (links per node – memory vs recall), ef_construction (build quality), ef_search (query accuracy vs speed). Complexity ≈ O(log N) per query; high recall (>95%) with millisecond latency; cost: high memory, slower inserts, deletes are tricky.

Q8. Compare vector databases: Chroma, FAISS, Pinecone, Qdrant, Weaviate, Milvus, pgvector.

Answer

- **Chroma** – open-source, embedded/simple; great for **prototyping** and small apps; persists to local disk.
- **FAISS** (Meta) – a **library**, not a database; extremely fast, many index types (Flat, IVF, PQ, HNSW), GPU support; no built-in persistence server, metadata filtering or CRUD API.
- **Pinecone** – fully **managed, serverless** cloud vector DB; easy scaling, metadata filtering, hybrid search; paid, data in vendor cloud.

- **Qdrant** – open-source (Rust), fast, rich payload filtering, hybrid search, self-host or cloud.
- **Weaviate** – open-source, built-in vectorization modules, hybrid (BM25 + vector) search, GraphQL API.
- **Milvus / Zilliz** – open-source, built for **billions of vectors**, distributed, GPU indexes.
- **pgvector** – extension that adds vector search to **PostgreSQL**; use when you already use Postgres and want SQL + vectors + transactions in one place; good up to millions of vectors.

(Also Elasticsearch/OpenSearch, Redis, MongoDB Atlas Vector Search, LanceDB.) Choose by scale, filtering needs, hosting/privacy, existing stack and cost.

Q9. Store embeddings in ChromaDB, persist them to disk and load them back. [CODE]

Answer

```
# pip install langchain-chroma langchain-openai
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 1) Create + persist to disk
vectordb = Chroma.from_documents(documents=chunks, embedding=embeddings,
                                 collection_name="policies",
                                 persist_directory="./chroma_db")   # auto-saved

# 2) Load later from disk (same embedding model!)
vectordb = Chroma(collection_name="policies", embedding_function=embeddings,
                  persist_directory="./chroma_db")

vectordb.add_documents(new_chunks)                # add more later
print(vectordb.similarity_search("What is the refund policy?", k=3))
```

(Older versions needed `vectordb.persist()`; with chromadb ≥ 0.4 data is persisted automatically.)

Q10. Create retrievers that fetch: (a) top-5 records, (b) records with similarity score > 0.5, (c) results using MMR. [CODE]

Answer

```
# (a) Top-5 most similar
retriever_top5 = vectordb.as_retriever(search_type="similarity", search_kwargs={"k": 5})

# (b) Only results with relevance score > 0.5
retriever_thresh = vectordb.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.5, "k": 10})

# (c) MMR - relevant AND diverse
retriever_mmr = vectordb.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 5, "fetch_k": 20, "lambda_mult": 0.5})

docs = retriever_top5.invoke("How do I reset my password?")
for d in docs:
    print(d.metadata.get("source"), d.page_content[:80])

# Scores directly
vectordb.similarity_search_with_relevance_scores("reset password", k=5)

# Metadata filter
vectordb.as_retriever(search_kwargs={"k": 5, "filter": {"department": "HR"}})
```

Q11. What is MMR (Maximal Marginal Relevance) and why use it? ★

Answer

Maximal Marginal Relevance (MMR) selects results that are **relevant to the query but different from each other**.

Algorithm: fetch `fetch_k` candidates (e.g. 20) by similarity, then pick results one by one, each time choosing the candidate that maximizes:

$\text{MMR} = \lambda \cdot \text{sim}(\text{query}, \text{doc}) - (1 - \lambda) \cdot \max \text{sim}(\text{doc}, \text{already_selected})$

$\lambda = 1 \rightarrow$ pure relevance; $\lambda = 0 \rightarrow$ maximum diversity (`lambda_mult` in LangChain).

Why use it: plain top-k often returns **near-duplicate chunks** (same paragraph from several versions of a document), wasting context-window space. MMR gives broader coverage of different aspects of the question.

Q12. What is hybrid search (BM25 + vector)? Why does it help? ★

Answer

Hybrid search combines:

- **Keyword/lexical search (BM25)** – exact term matching; excellent for **names, IDs, codes, acronyms, rare terms** ("error E-4021", "Form 16", product SKUs).
- **Vector/semantic search** – matches **meaning and paraphrases** ("how to get money back" ↔ "refund policy").

Results are merged using **Reciprocal Rank Fusion (RRF)** or a weighted score: score = α·vector + (1 − α)·BM25.

Why it helps: each method covers the other's weakness – embeddings miss exact rare tokens, BM25 misses synonyms – so recall improves noticeably.

```
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever   # langchain<1.0: langchain.retrievers
bm25 = BM25Retriever.from_documents(chunks); bm25.k = 5
hybrid = EnsembleRetriever(retrievers=[bm25, vectordb.as_retriever(search_kwargs={"k": 5})],
                           weights=[0.4, 0.6])
```

Q13. What is re-ranking? Explain cross-encoders and Cohere Rerank. ★

Answer

Re-ranking is a second stage after retrieval: retrieve a larger candidate set quickly (e.g. top-25 via vectors/BM25), then use a more accurate model to **re-score each (query, document) pair** and keep the best few (top-3–5) for the LLM.

- **Bi-encoder** (embedding model): encodes query and document **separately** → fast, can pre-compute document vectors, but less precise.
- **Cross-encoder**: feeds **query and document together** into one Transformer, which outputs a relevance score – it sees full word-level interaction, so it is **much more accurate** but too slow to run over the whole corpus. Examples: ms-marco-MiniLM cross-encoders, **bge-reranker**, Jina reranker.
- **Cohere Rerank** – a hosted, multilingual cross-encoder re-ranking API (CohereRerank in LangChain).

```
from langchain_classic.retrievers import ContextualCompressionRetriever   # langchain<1.0:
    langchain.retrievers
from langchain_cohere import CohereRerank
reranker = CohereRerank(model="rerank-english-v3.0", top_n=4)
rerank_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=vectordb.as_retriever(search_kwargs={"k": 25}))
```

Re-ranking typically gives one of the biggest accuracy gains in RAG.

Q14. Explain MultiQueryRetriever, ContextualCompressionRetriever, ParentDocumentRetriever and SelfQueryRetriever.

Answer

- **MultiQueryRetriever** – uses an LLM to **rewrite the user question into several different queries** (different wording/perspectives), retrieves for each, and returns the **unique union** of results. Overcomes the sensitivity of vector search to exact phrasing.
- **ContextualCompressionRetriever** – wraps a base retriever and **compresses/filters the retrieved documents** using a compressor: an LLM that extracts only the relevant sentences (LLMChainExtractor), an embeddings filter that drops low-similarity docs, or a **re-ranker**. Less noise and fewer tokens in the prompt.
- **ParentDocumentRetriever** – indexes **small child chunks** (precise embeddings) but returns their **larger parent chunk/document** (full context) to the LLM. Needs a vector store (children) and a docstore (parents).
- **SelfQueryRetriever** – uses an LLM to turn a natural-language question into a **semantic query + metadata filter**. "Movies about aliens released after 2015 with rating above 8" → query "aliens", filter year > 2015 AND rating > 8. Needs metadata field descriptions.

```
from langchain_classic.retrievers.multi_query import MultiQueryRetriever   # langchain<1.0:
    langchain.retrievers.multi_query
mq = MultiQueryRetriever.from_llm(retriever=vectordb.as_retriever(), llm=llm)
```

Q15. What are query transformation techniques (HyDE, query rewriting, query decomposition)?

Answer

- **Query rewriting** – an LLM rewrites a vague/conversational question into a clear standalone search query (e.g. resolve "what about its price?" using chat history).
- **Multi-query** – generate several paraphrases and merge the results.
- **HyDE (Hypothetical Document Embeddings)** – the LLM first writes a **hypothetical answer**, which is embedded and used for search; a fake answer is often closer in embedding space to real answer passages than the short question is.
- **Query decomposition** – split a complex question into sub-questions ("Compare the leave policy of 2023 and 2024" → two queries), retrieve for each, then combine.
- **Step-back prompting** – ask a more general question first to retrieve background principles.
- **Query routing** – send the query to the right index/data source (SQL vs documents vs web).

Q16. Build a complete retrieval chain (retriever + prompt + LLM) that cites its sources. [CODE]

Answer

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
retriever = vectordb.as_retriever(search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context below. Cite sources as [source, page]. "
    "If the answer is not in the context, say 'I don't know'.\n\n"
    "Context:\n{context}\n\nQuestion: {question}")

def format_docs(docs):
    return "\n\n".join(f"[{d.metadata.get('source')}, p{d.metadata.get('page')}] {d.page_content}"
                       for d in docs)

answer_chain = (RunnablePassthrough.assign(context=lambda x: format_docs(x["docs"]))
                | prompt | llm | StrOutputParser())

rag_chain = (RunnableParallel(docs=retriever, question=RunnablePassthrough())
             .assign(answer=answer_chain))

out = rag_chain.invoke("What is the notice period for resignation?")
print(out["answer"])
print({d.metadata["source"] for d in out["docs"]})      # sources used
```

(Legacy helpers, now in langchain_classic.chains: `create_retrieval_chain(retriever, create_stuff_documents_chain(llm, prompt))` returns {"answer", "context"}.)

Q17. How do you build RAG using an open-source LLM end-to-end?

Answer

Replace each closed component with an open-source one – everything can run locally/on-prem:

- **LLM:** Llama 3.x, Mistral, Qwen, Gemma via **Ollama** (`ChatOllama`), **vLLM** (OpenAI-compatible server) or Hugging Face pipelines; quantized (4-bit GGUF/AWQ) to fit the GPU.
- **Embeddings:** BGE / E5 / nomic-embed via `HuggingFaceEmbeddings` or Ollama embeddings.
- **Vector DB:** Chroma, FAISS, Qdrant, Milvus, pgvector.
- **Re-ranker:** bge-reranker (cross-encoder).

```
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma
llm = ChatOllama(model="llama3.1:8b", temperature=0)
emb = OllamaEmbeddings(model="nomic-embed-text")
vectordb = Chroma.from_documents(chunks, emb, persist_directory="./db")
# then the same LCEL RAG chain as in the previous question
```

Use the model's correct **chat template**, keep prompts short and explicit (smaller models follow instructions less reliably), and evaluate with RAGAS.

Q18. How do you feed table data and image data to a RAG system?

Answer

Tables:

- Parse with table-aware tools (Unstructured, **pdfplumber/Camelot**, LlamaParse, Docling, Azure Document Intelligence) instead of plain text extraction, which scrambles rows.
- Convert each table to **Markdown/HTML/JSON** and keep it as **one chunk** (do not split tables); add the table title and column headers to every chunk if a table must be split.
- Store an **LLM-generated summary** of the table for retrieval, and return the full table to the LLM (multi-vector retriever).
- For large structured data (CSV/Excel/DB), use **Text-to-SQL** or a pandas agent instead of embeddings.

Images:

- Use a **vision-language model** to generate **captions/descriptions** of images, charts and diagrams; embed and index the text, keep a link to the original image, and send the image itself to a multimodal LLM at answer time.
- Or use **multimodal embeddings** (CLIP, ColPali for page images) to embed images and text in the same space.
- OCR for scanned documents/text inside images.

Q19. What is Agentic RAG? What is GraphRAG? ★

Answer

Agentic RAG – instead of a fixed "retrieve once, then answer" pipeline, an **LLM agent controls retrieval**: it decides **whether** to retrieve, **which source/tool** to use (vector DB, SQL, web search, API), **rewrites queries**, **grades the retrieved documents** for relevance, retrieves again if needed, and checks its answer for hallucination before responding. Patterns: Self-RAG, Corrective RAG (CRAG), adaptive RAG – typically built with **LangGraph**. Better for complex multi-step questions; slower and costlier.

GraphRAG (popularized by Microsoft) – uses an LLM to extract **entities and relationships** from documents into a **knowledge graph**, detects communities and generates summaries for them. Retrieval then follows graph relationships and community summaries. Good for **multi-hop questions** ("Which suppliers are linked to customers who reported fraud?") and **global questions** over the whole corpus ("What are the main themes?") that plain chunk retrieval answers badly. Tools: Microsoft GraphRAG, Neo4j + LangChain, LlamaIndex KG index.

Q20. Your RAG system gives wrong answers. How do you debug it step by step (retrieval vs generation)? ★

Answer

Debug the two halves separately using logs/traces (LangSmith/Langfuse) and a test set of questions with known answers:

1. Check retrieval first – for each failing question, look at the retrieved chunks. Is the correct passage in the top-k?

- **Not retrieved** → data issue (document missing, parsing broke tables/PDF text), bad chunking (answer split across chunks), weak embedding model for the domain/language, query phrasing mismatch, k too small, missing metadata filters. Fixes: better parsing, chunk size/overlap, hybrid search (BM25), query rewriting/multi-query, re-ranker, domain embeddings.
- Measure with **context recall / precision, hit rate, MRR**.

2. If retrieval is correct but the answer is wrong → generation problem:

- Prompt not strict enough ("answer only from context"), too much noisy context, lost-in-the-middle, model too weak, temperature too high, conflicting/outdated documents. Fixes: better prompt, fewer but better chunks (re-ranking), reorder chunks, stronger model, ask for citations.
- Measure **faithfulness and answer relevancy** (RAGAS / LLM-as-judge).

3. Add failing cases to the regression test set, fix, and re-evaluate.

Q21. Explain some real-world RAG use cases.

Answer

- **Enterprise knowledge assistant** – Q&A over HR policies, SOPs, wikis, Confluence.
- **Customer-support chatbot** grounded in product manuals and FAQs, with ticket deflection.
- **Banking/finance** – answering questions on regulatory circulars (RBI guidelines), credit policies; assisting fraud analysts by retrieving similar past cases.
- **Legal & compliance** – contract search, clause comparison.
- **Healthcare** – clinical guideline lookup for doctors.
- **Developer assistants** – Q&A over codebases and API docs.
- **Research & analyst tools** – summarizing and querying reports, earnings calls, papers with citations.
- **E-commerce** – product Q&A and recommendations.